page, an image, a JavaScript file, a simple acknowledgment, or practically any other file. For this example, I'll assume that the user has reached the home page of a web app, so the server responds with some HTML. The HTML is added to the HTTP response, which is sent back across the internet to the browser that made the request.

As soon as the user's browser begins receiving the HTTP response, it can start displaying content on the screen, but the HTML page may also reference other pages and links on the server. To display the complete web page instead of a static, colorless, raw HTML file, the browser must repeat the request process, fetching every referenced file. HTML, images, Cascading Style Sheets (CSS) for styling, and JavaScript files for extra behavior are all fetched using exactly the same HTTP request process.

Pretty much all interactions that take place on the internet are a facade over this basic process. A basic web page may require only a few simple requests to render fully, whereas a large modern web page may take hundreds. At this writing, the Amazon .com home page (https://www.amazon.com) makes 410 requests, including requests for 4 CSS files, 12 JavaScript files, and 299 image files!

Now that you have a feel for the process, let's see how ASP.NET Core dynamically generates the response on the server.

1.4.2 How does ASP.NET Core process a request?

When you build a web application with ASP.NET Core, browsers will still be using the same HTTP protocol as before to communicate with your application. ASP.NET Core itself encompasses everything that takes place on the server to handle a request, including verifying that the request is valid, handling login details, and generating HTML.